

# Technical Report: Student Records Server Security, Cryptography, and Firewall Implementation

**Richard MAZIMPAKA**  
(Roll number: 4202670003)  
System Security Administrator

September 27, 2026

## Asset Valuation, Vulnerabilities, Risk Rankings, and Controls

Protecting institutional data requires identifying core system assets, evaluating potential attack vectors, assigning risk rankings based on likelihood/impact matrices, and applying robust mitigation controls.

| Asset                                             | Vulnerability                                                      | Risk Rank | Recommended Control                                                                                  |
|---------------------------------------------------|--------------------------------------------------------------------|-----------|------------------------------------------------------------------------------------------------------|
| Student Records File<br>(student_reco<br>rds.txt) | Unauthorized local tampering or data theft via plaintext exposure. | High      | Symmetric encryption (Fernet) and streaming SHA-256 integrity baseline hashing.                      |
| Cryptographic Key<br>(secret.key)                 | Key leakage or improper file permission disclosures.               | Critical  | Enforce strict owner-only file permissions (0o600) and exclude completely from git version tracking. |
| Service Port 443<br>(HTTPS Binding)               | Unauthorized inbound access from untrusted or guest subnets.       | High      | Stateful packet filtering via iptables rules blocking guest/external networks.                       |

## Cryptographic and Integrity Functions

The companion application handles data security constraints through the cryptography .fernet standard symmetric envelope architecture alongside stateful streaming hashes.

## Program Execution and Real-World Evidence Logs

The core software framework was invoked inside the student records ecosystem environment. Below is the verified actual execution command sequence tracking encryption routines, plain-text byte matching confirmations, and pristine cryptographic verification hashes:

```
PS C:\Users\User\Desktop\Exam> & C:\Users\User\AppData\Local\Programs\Python\Python311\python.exe  
c:/Users/User/Desktop/Exam/encryp_student_records.py
```

```
=====
  ENCRYPTION & INTEGRITY APPLICATION
=====
1. [Part a] Encrypt student record file
2. [Part b] Decrypt file and verify match
3. [Part c] Perform SHA-256 integrity audit check
4. [Simulate] Unauthorized file tampering modification
5. Exit application
=====
Select an option (1-5): 1
[✓] Successfully encrypted data into: student_records.enc
[✓] Baseline SHA-256 integrity hash tracked:
0304e5d73b70a4cc821cc774df21db6614a987c30dcb1cdf135e2437121902

Select an option (1-5): 2
[✓] Data decrypted and saved to: student_records_decrypted.txt
[✓] VERIFICATION SUCCESS: Decrypted contents match the original
precisely.

Select an option (1-5): 3
Stored Baseline Hash:
0304e5d73b70a4cc821cc774df21db6614a987c30dcb1cdf135e2437121902
Current File Hash:
0304e5d73b70a4cc821cc774df21db6614a987c30dcb1cdf135e2437121902
[✓] INTEGRITY PASSED: The file remains authentic and unchanged.
```

## Firewall Architecture, Configurations, and Boundary Verification

Network topology layers were evaluated against explicit iptables filtering frameworks targeting Port 443 bindings over 192.168.1.174 interfaces.

## Core Stateful Implementation Mapping

To guarantee staff elements retain stable continuous operational flow vectors without being systematically trapped by subsequent macro blocks, stateful connection filters (ESTABLISHED, RELATED) are placed first in the engine queue structure.

## Network Environment Infrastructure Verification Terminal Evidence

The system was run through real-world client interaction pipelines to explicitly demonstrate rule exceptions. The logs below reveal immediate HTTP 200 payload returns from the authorized staff domain alongside full directory lists:

```
C:\Users\User>%USERPROFILE%\AppData\Local\Programs\Python\Python311\python.exe -m http.server 443 --bind 0.0.0.0
Serving HTTP on 0.0.0.0 port 443 (http://0.0.0.0) ...
192.168.152.1 - - [27/Sep/2026 14:39:05] "GET / HTTP/1.1" 200 - C:\Users\User>curl http://192.168.152.1:443 --connect-timeout 5
<!DOCTYPE HTML>
<html lang="en">
<head><meta charset="utf-8"/><title>Directory listing for /</title></head>
<body><h1>Directory listing for /</h1><hr><ul>
<li><a href=".anaconda/">.anaconda/</a></li><li><a href=".arduinoIDE/">.arduinoIDE/</a></li>
<li><a href=".conda/">.conda/</a></li><li><a href=".vscode/">.vscode/</a></li>
<li><a href="AppData/">AppData/</a></li><li><a href="Desktop/">Desktop/</a></li>
<li><a href="Documents/">Documents/</a></li><li><a href="Downloads/">Downloads/</a></li>
</ul><hr></body></html>
```

## Conclusion and Repository Matrix Coordinates

The configurations successfully shrink systemic institutional vector surfaces down by isolating high-value databases behind authenticated codebases and granular stateful ingress controls.

### GitHub Project Production Link:

<https://github.com/richardmazimpk/cryptography-network-security-exam>